

Diseño y Desarrollo de Servicios Web-Proyecto

Jhon Ramiro Zapata Pérez

Instructor

Andrés Montoya

Centro de la Tecnología del Diseño y la Productividad Empresarial Regional

Cundinamarca sede Girardot

Análisis y Desarrollo de Software

Agosto de 2026

Caloto-Cauca

Introducción

En el desarrollo de aplicaciones web modernas es fundamental separar la lógica del negocio de la interfaz que utiliza el usuario final. Este proyecto implementa dicha separación mediante arquitectura cliente-servidor: por un lado, una API construida en Node.js con el framework Express(server.js), encargada de gestionar el registro, la autenticación y la administración de usuarios sobre una base de datos PostgreSQL; y por otro lado, una página web (registroInicio.html) que actúa como cliente y consume dicha API mediante peticiones HTTP en formato JSON. A través de este documento se explica en detalle el funcionamiento interno de la API, sus distintos endpoints, validaciones y medidas de seguridad implementadas, así como el mecanismo mediante el cual el cliente web se comunica con ella para ofrecer al usuario una experiencia de registro e inicio de sesión funcional.

Objetivo.

Explicar de manera clara y detallada el funcionamiento de la API desarrollada en el archivo server.js, así como el proceso mediante el cual esta es consumida por el cliente web registroInicio.html. Describiendo sus endpoints, validaciones, medidas de seguridad y flujo completo de comunicación entre el frontend y backend, con el fin de evidenciar la comprensión del funcionamiento de una API REST dentro de una arquitectura cliente-servidor.

Arquitectura general del sistema.

El flujo general de comunicación sigue el patrón cliente-servidor típico de una arquitectura REST.

```

Usuario (navegador)
  |
  v
registroInicio.html --(HTML/CSS)--> Interfaz visual
  |
  | fetch() -> Peticion HTTP (JSON)
  v
API Express (server.js) --puerto 8080--
  |
  | Validaciones + bcrypt (hash de contraseñas)
  v
Base de datos PostgreSQL
  tabla: public."UsuariosRegistrados"

```

Tecnologías y dependencias utilizadas.

Dependencia	Función dentro del proyecto
express	Framework que crea el servidor HTTP y define las rutas (endpoints) de la API.
pg	Cliente de PostgreSQL para Node.js; permite ejecutar consultas SQL (Pool de conexiones).
bcryptjs	Genera un hash seguro de la contraseña y permite compararla en el login, sin guardar la contraseña en texto plano.
cors	Habilita las peticiones desde un origen distinto (el archivo HTML abierto en el navegador) hacia la API.
dotenv	Carga variables de entorno (como DATABASE_URL) desde el archivo .env, evitando exponer credenciales en el código.

El servidor se pone en marcha con `app.listen(8080)`, por lo que toda la API vive en la dirección base <http://localhost:8080>.

Endpoints de la API (server.js)

La API expone cinco rutas (endpoints). A continuación, se explica cada una: qué recibe, qué valida, qué hace en la base de datos y qué responde.

- **POST /sign-up → Registrar un nuevo usuario.**

Método / Ruta	POST /sign-up
Datos que recibe (body JSON)	nombre, edad, correo, contraseña
Validaciones	Que existan nombre, edad, correo y contraseña; que la edad no sea menor a 17 ni mayor a 70. Si algo falla, responde 400 con un mensaje de error.
Seguridad	La contraseña nunca se guarda tal cual: se genera una "sal" con <code>bcrypt.genSalt(10)</code> y luego se calcula un hash con <code>bcrypt.hash()</code> . Ese hash es lo único que se guarda en la base de datos.
Acción en la base de datos	INSERT en la tabla <code>public."UsuariosRegistrados"</code> con nombre, edad, correo y el hash de la contraseña.
Respuesta exitosa	201 Created → { "message": "Usuario Registrado!" }
Posibles errores	400 Bad Request con mensajes como "El nombre no ha sido ingresado", "No cuenta con la edad suficiente", "Edad fuera del rango maximo", entre otros.

- **POST /sign-in → Iniciar sesión.**

Método / Ruta	POST /sign-in
Datos que recibe (body JSON)	correo, contraseña

Validaciones	Que se envíe correo y contraseña (400 si falta alguno).
Lógica principal	Busca en la tabla "UsuariosRegistrados" un registro con ese correo (SELECT). Si no existe, responde 401 "El usuario no existe". Si existe, compara la contraseña recibida contra el hash guardado usando bcrypt.compare().
Respuesta exitosa	200 OK → { "message": "Bienvenido!" }
Posibles errores	400 si faltan datos; 401 "El usuario no existe" o 401 "Contraseña incorrecta" si la comparación de bcrypt falla.

- **GET /UsuariosRegistrados → Consultar usuarios.**

Recibe parámetros por query string (?nombre= . . . o?correo= . . .).

Exige al menos uno de los dos. Busca coincidencias exactas en la base de datos y antes de responder elimina el campo contraseña de cada resultado por seguridad. Devuelve 200 con la lista de usuarios (sin contraseñas) o 400 si no hay coincidencias o no se envió ningún criterio.

- **PUT /UsuariosRegistrados/:id → Actualizar un usuario**

Recibe el id del usuario por la URL y, en el body, cualquier combinación de nombre, edad, correo y/o contraseña (todos opcionales). Construye dinámicamente la sentencia SQL UPDATE, agregando solo los campos que realmente llegaron. Si se envía una nueva contraseña, también se vuelve a encriptar con bcrypt antes de guardarla. Valida que el id sea numérico; responde 404 si el usuario no existe, y 200 con el usuario actualizado (sin la contraseña) si todo sale bien.

- **DELETE /UsuariosRegistrados/:id → Eliminar a un usuario.**

Recibe el id por la URL y ejecuta un DELETE sobre la tabla. Si no se elimina ninguna fila, responde 404 "El usuario no existe"; si tiene éxito, responde 200 con el mensaje "Usuario eliminado!".

Nota: El archivo registroInicio.html solo consume los endpoints /sign-up y /sign-in. Las rutas GET, PUT y DELETE existen en la API para administrar usuarios (por ejemplo desde Postman o Insomnia), pero no tienen un formulario asociado en la interfaz actual.

Middlewares configurados en la API.

- `app.use(express.json())`: permite que Express lea automáticamente el cuerpo (body) de las peticiones cuando viene en formato JSON, y lo convierte en un objeto JavaScript accesible en `request.body`.
- `app.use(cors())`: habilita el intercambio de recursos entre orígenes distintos (Cross-Origin Resource Sharing). Es indispensable porque registroInicio.html se abre normalmente como un archivo local o desde otro origen distinto a `localhost:8080`; sin CORS, el navegador bloquearía las peticiones fetch por política de seguridad.

Cómo consume la API el archivo registroInicio.html

RegistroInicio.html es una página estática con dos formularios (registro e inicio de sesión) y un bloque de Javascript que se encarga de comunicarse con la API mediante la función `fetch()`. A continuación, se explica paso a paso.

- **Definición de la dirección base de la API**

```
const API = "http://localhost:8080";
```

Toda petición que haga el cliente se construye concatenando esta constante con la ruta específica del endpoint (por ejemplo, API + "/sign-up").

- **Estructura HTML de los formularios**

El documento define dos formularios con ID form_registro y form_login. Cada input tiene un atributo name (nombre, edad, correo, contraseña) que coincide exactamente con las claves que la API espera recibir en el body. Esta coincidencia de nombres es clave porque permite construir el objeto de datos automáticamente sin tener que leer campo por campo.

- **Función genérica enviar (ruta, datos)**

Todo el consumo de la API pasa por una única función reutilizable, encargada de enviar la petición y de interpretar la respuesta:

```
async function enviar(ruta, datos) {  
  try {  
    const respuesta = await fetch(API + ruta, {  
      method: "POST",  
      headers: { "Content-Type": "application/json" },  
      body: JSON.stringify(datos),  
    });  
  
    const cuerpo = await respuesta.json();  
  
    if (!respuesta.ok) {  
      return mostrar(cuerpo.error, true);  
    }  
  }  
}
```

```

mostrar(cuerpo.message, false);
} catch (error) {
  mostrar("No se pudo conectar con el servidor", true);
}
}

```

Su funcionamiento paso a paso:

1. **method: "POST"** → Indica que se va a enviar información al servidor (coincide con `app.post` en la API).
2. **headers** → Declara `Content-Type application/json` para que Express (gracias a `express.json()`) interprete correctamente el cuerpo recibido.
3. **body: JSON.stringify(datos)** → Convierte el objeto JavaScript con los datos del formulario en texto JSON para poder enviarlo por HTTP.
4. **respuesta.json()** → Toma la respuesta que devuelve la API (también es JSON) y la convierte de nuevo en un objeto JavaScript.
5. **respuesta.ok** → Es verdadero solo si el código de estado HTTP está entre 200 y 299. Si la API respondió con 400, 401, 404 o 500, `respuesta.ok` es falso y se muestra como `cuerpo.error` en pantalla.
6. **catch (error)** → Captura fallas de red (por ejemplo, que la API no esté encendida) y muestra el mensaje "No se pudo conectar con el servidor".

Captura de los formularios (submit)

Cada formulario tiene un listener sobre el evento `submit`. Ambos siguen el mismo patrón: evitan que la página se recargue, leen los datos del formulario con `FormData` y los

convierten en un objeto plano con `Object.fromEntries()`, y finalmente llaman a `enviar()` con la ruta correspondiente.

```
document.getElementById("formRegistro").addEventListener("submit", (evento) => {
  evento.preventDefault();
  const datos = Object.fromEntries(new FormData(evento.target));
  datos.edad = Number(datos.edad);
  enviar("/sign-up", datos);
});

document.getElementById("formLogin").addEventListener("submit", (evento) => {
  evento.preventDefault();
  const datos = Object.fromEntries(new FormData(evento.target));
  enviar("/sign-in", datos);
});
```

En el formulario de registro se hace una conversión adicional: `datos.edad = Number(datos.edad)`, porque todos los valores que entrega `FormData` llegan como texto (string), y la API en `server.js` necesita comparar la edad numéricamente (`edad > 0 && edad < 17`).

Visualización del resultado

La función `mostrar (texto, esError)` escribe el mensaje recibido dentro del elemento `<div="mensaje">` y le agrega a clase CSS “error” u “ok” según corresponda, lo que permite que `style.css` cambie su color (por ejemplo, rojo para errores y verde para éxito).

Flujo completo de una petición (ejemplo: registro)

7. El usuario llena el formulario de registro y presiona "Crear cuenta".
8. El evento submit se intercepta y se arma el objeto {nombre, edad, correo, contraseña}.
9. Se llama a `enviar("/sign-up", datos)`, que hace fetch a <http://localhost:8080/sign-up> con method POST.
10. La API recibe la petición en `app.post("/sign-up", ...)`, valida los campos y la edad.
11. Si todo es válido, encripta la contraseña con bcrypt y ejecuta un INSERT en PostgreSQL.
12. La API responde con estado 201 y { "message": "Usuario Registrado!" }.
13. El navegador recibe la respuesta, `respuesta.ok` es verdadero y se muestra el mensaje de éxito en la pantalla mediante `mostrar()`.

El flujo de inicio de sesión (/sign-in) es idéntico en su mecánica, cambiando únicamente los datos enviados (correo y contraseña) y la validación interna, que en vez de insertar un registro, busca el usuario y compara la contraseña con `bcrypt.compare()`.

Resumen de la relación API-Cliente.

Formulario en el HTML	Endpoint que consume	Resultado esperado
formRegistro	POST /sign-up	Usuario creado en la base de datos con contraseña encriptada.

Formulario en el HTML	Endpoint que consume	Resultado esperado
formLogin	POST /sign-in	Validación de credenciales y mensaje de bienvenida.

En conjunto, registroInicio.html funciona como la capa de presentación (frontend) y server.js como la capa de lógica de negocio y persistencia (backend/API). La comunicación entre ambos se realiza exclusivamente mediante peticiones a HTTP en formato JSON, siguiendo el modelo cliente-servidor propio de una API REST.

Conclusión.

El desarrollo de este proyecto permitió evidenciar cómo una API construida con Node.js y Express puede gestionar de forma segura y organizada operaciones como el registro, la autenticación y la administración de usuarios, aplicando buenas prácticas como cifrar contraseñas con bcrypt, validar datos de entrada y usar una base de datos relacional para la persistencia de la información.

Así mismo, se comprobó que la comunicación entre el cliente (registroInicio.html) y la API se realiza de manera desacoplada mediante peticiones HTTP en formato JSON, gestionadas con Fetch, Esto demuestra la aplicación práctica del modelo cliente-servidor y de una arquitectura tipo REST.

En conjunto, el proyecto refleja conocimientos adquiridos sobre el consumo e implementación de APIs, así como su integración con interfaces web.